

## Feb 9th HW 16

### 1. What are the three pillars of observability, and what is the primary purpose of each?

- The three pillars of observability are Metrics, Logs, and Traces.
  - Metrics provide numerical data over time, such as CPU usage or request rate. Their primary purpose is monitoring system health and triggering alerts.
  - Logs provide detailed event records generated by applications or systems. They are mainly used for debugging and understanding specific events.
  - Traces track the flow of a request across multiple services in distributed systems. Their primary purpose is identifying latency bottlenecks and service dependencies.
- Together, these three pillars allow engineers to understand system behavior from high-level health monitoring to deep debugging analysis.

### 2. Explain the four common metric types and give an example use case for each.

- There are four common metric types:
  - Counter – A cumulative value that only increases (e.g., total HTTP requests processed).
  - Gauge – A value that can increase or decrease (e.g., current memory usage).
  - Histogram – Measures distribution of values into buckets (e.g., request latency distribution).
  - Summary – Similar to histogram but calculates quantiles (e.g., 95th percentile response time).
- Each type serves a different monitoring purpose, especially in systems like Prometheus.

### 3. What is structured logging, and why is it preferred over unstructured (plain text) logging in production systems?

- Structured logging records logs in a consistent, machine-readable format such as JSON rather than plain text. For example:
  - ```
{  
  "timestamp": "2026-02-10T10:00:00Z",  
  "level": "ERROR",  
  "service": "payment-service",  
  "message": "Payment failed",  
  "userId": 12345  
}
```
- Structured logging is preferred in production because it allows efficient searching, filtering, indexing, and aggregation in log management systems like ELK or Splunk. Plain text logs are harder to parse and analyze at scale.

**4. Explain the relationship between Trace, Span, and Context Propagation in distributed tracing.**

- In distributed tracing:
  - A Trace represents the entire lifecycle of a request across multiple services.
  - A Span represents a single operation within a trace (e.g., a database query or API call).
  - Context Propagation ensures trace information (trace ID, span ID) is passed between services, usually via HTTP headers.
- Each trace consists of multiple spans linked together. Context propagation ensures all services involved in a request are part of the same trace, enabling full request visibility.

**5. Prometheus uses a "pull" model for collecting metrics. Explain how this works and what are its advantages and disadvantages compared to a "push" model.**

- Prometheus uses a pull model, meaning it periodically scrapes metrics from target endpoints (usually /metrics).
- How it works:
  - Applications expose metrics via HTTP endpoints.
  - Prometheus server polls these endpoints at regular intervals.
- Advantages:
  - Simple architecture.
  - Easier service discovery.
  - Prometheus controls collection frequency.
  - Better reliability (central control).
- Disadvantages:
  - Targets must be reachable.
  - Not ideal for short-lived batch jobs (push model may work better here).
- Compared to push models (e.g., StatsD), pull gives more centralized control but less flexibility for edge cases.

**6. What are the four main components of the Kubernetes control plane, and what is the responsibility of each?**

- The four main control plane components are:
  - kube-apiserver – Entry point for all Kubernetes API requests. It validates and processes cluster operations.
  - etcd – Distributed key-value store that stores all cluster state.
  - kube-scheduler – Assigns Pods to nodes.
  - kube-controller-manager – Runs controllers that ensure cluster state matches desired state.
- Together, they manage the cluster's control logic.

**7. What is kubelet and what role does it play on a Kubernetes worker node?**

- kubelet is an agent running on each worker node. It:
  - Communicates with the API server.
  - Ensures containers defined in PodSpecs are running.
  - Reports node and pod status back to the control plane.
  - Works with the container runtime (containerd, CRI-O).
- It acts as the “node manager” in Kubernetes.

#### **8. What is etcd in Kubernetes, and why is it critical to the cluster's operation?**

- etcd is a distributed key-value database used by Kubernetes to store cluster configuration and state.
- It is critical because:
  - It stores all cluster objects (Pods, Services, ConfigMaps, Secrets).
  - If etcd fails or data is lost, the cluster cannot function properly.
  - It ensures consistency through strong consensus (Raft algorithm).
- Without etcd, Kubernetes loses its source of truth.

#### **9. What is kube-proxy and how does it enable Service networking in Kubernetes?**

- kube-proxy runs on each node and manages networking rules to enable communication with Kubernetes Services.
- It:
  - Maintains iptables or IPVS rules.
  - Enables load balancing across Pods behind a Service.
  - Allows stable virtual IP access to dynamically changing Pods.
- It ensures Service abstraction works properly.

#### **10. How does the Kubernetes Scheduler decide which node to place a Pod on? What factors does it consider?**

- The Kubernetes Scheduler decides where to place a Pod by evaluating:
  - Resource availability (CPU, memory).
  - Node selectors / affinity rules.
  - Taints and tolerations.
  - Topology constraints.
  - Pod priority and preemption.
  - Custom scheduling policies.
- The process includes:
  - Filtering phase – Remove unsuitable nodes.
  - Scoring phase – Rank remaining nodes.
  - Select the highest-scoring node.
- This ensures optimal resource utilization and policy compliance.